

SKILL CHECK ACTIVITY

Data Analyst Skill Check

Python · NumPy · Pandas · Matplotlib

A 45–90 minute scenario-based activity

Prepared for MSCW, DU · Final Year Students

HarborLine Cruises · Junior Data Analyst Simulation

DURATION

45–90 minutes

DATASET

Titanic (fixed)

SUBMISSION

Public GitHub repo

EVALUATION

Self-scored rubric

BEFORE YOU BEGIN

- 1 Read every section once, end to end, before writing any code.
- 2 Set a visible timer for 90 minutes — track your actual pace.
- 3 Work in a notebook you can push to a public GitHub repo.
- 4 Score yourself honestly using the rubric near the end.

Prepared exclusively for MSCW, DU final year students · Read this document fully before starting the timer

Prepared by Parth Shukla · github.com/ParthShuklaa

CONTENTS

→ Scenario

→ Dataset

→ Time Budget

→ Part 1 — Setup & Exploration (10 min)

→ Part 2 — NumPy Tasks (15 min)

→ Part 3 — Pandas Tasks (25 min)

→ Part 4 — Matplotlib Tasks (20 min)

→ Part 5 — Report Writing (10–20 min)

→ GitHub Submission Instructions

→ Self-Evaluation Rubric

→ Total Score & Self-Placement

Scenario

You’ve just joined **HarborLine Cruises** as a junior data analyst. The operations team has handed you historical passenger data and asked for a quick analysis before tomorrow’s planning meeting. They want to understand **who was on board, who survived incidents, and what patterns exist by class, age, and fare** — using the (famous) Titanic dataset as your working data.

Your manager doesn’t care about perfect code — they care whether you can go from raw data to a clear, evidence-backed report **in under 90 minutes**. This assignment simulates that pressure test.

You will work in a Jupyter/Colab notebook, push it to a public GitHub repository, and self-score your own submission against the rubric in this document before sharing it.

Dataset

Use exactly this dataset — no substitutions. Everyone works from the same data so results are comparable across participants.

```
import pandas as pd
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)
```

Time Budget

Phase	Time	Focus
Setup & exploration	10 min	Load data, inspect structure, note issues
NumPy tasks	15 min	Vectorized calculations, array operations
Pandas tasks	25 min	Cleaning, filtering, grouping, feature engineering
Matplotlib tasks	20 min	Visualizing findings
Report & GitHub push	10-20 min	Write findings, commit, submit link

Total: 45-90 minutes. Don’t aim for perfection — aim for honest completion within the window. Note your actual time taken; you’ll report it at submission.

Part 1 — Setup & Exploration (10 min)

1. Load the dataset and display the first 5 rows.
2. Print `.info()` and `.describe()`.
3. Identify and write down **3 data quality issues** you notice (e.g. missing values, skewed distributions, mixed types).

Part 2 — NumPy Tasks (15 min)

Use NumPy directly (not pandas built-ins) wherever an array operation is asked:

1. Convert the `Age` and `Fare` columns to NumPy arrays.
2. Compute mean, median, and standard deviation of `Fare` using NumPy functions, ignoring NaNs.
3. Create a boolean mask array identifying passengers under 12 years old.
4. Use `np.where` to create an array labeling each passenger `"child"`, `"adult"`, or `"unknown"` (for missing age).
5. Replace missing values in the `Age` array with the column mean using `np.where` (not `.fillna()`).

Part 3 — Pandas Tasks (25 min)

1. Handle missing data: fill or drop `Age` , `Embarked` , and `Cabin` with a justified strategy (write one line explaining your choice for each column).
2. Create two engineered columns:
 - `FamilySize` = `SibSp` + `Parch` + 1
 - `FareCategory` = `"Low"` / `"Medium"` / `"High"` using `pd.cut` or custom logic
3. Answer these three business questions and print the result for each:
 - What was the survival rate by passenger class **and** gender combined?
 - Did larger families (`FamilySize` > 4) survive at a different rate than smaller families?
 - Which embarkation port had the highest average fare?
4. Produce one pivot table summarizing survival rate by `Pclass` and `AgeGroup` (bucket `Age` into child / teen / adult / senior first).

Part 4 — Matplotlib Tasks (20 min)

Create the following four charts, each with a title and axis labels:

1. **Histogram** — distribution of `Age`
2. **Bar chart** — survival rate by passenger class
3. **Scatter plot** — `Age` vs `Fare` , marked or colored by survival status
4. **Heat map** (Seaborn permitted) — correlation matrix of numeric columns

Save all four charts as `.png` files in a `charts/` folder in your repository.

Part 5 — Report Writing (10-20 min)

Write a short findings report — either as markdown cells in your notebook or a separate `REPORT.md` — covering:

1. **Top 3 insights** from your analysis, each backed by a specific number or chart.
2. **One recommendation** for the operations team based on the data.
3. **What you'd investigate next** if you had another hour.

GitHub Submission Instructions

Step 1 — Create your repository

1. Create a **new public repository** on GitHub named something like `harborline-skill-check`.
2. Do not fork or reuse a private repo — reviewers need direct public access.

Step 2 — Required folder structure

Match this structure exactly so submissions are easy to review side by side:

```
your-repo-name/
├─ notebook.ipynb          (or .py if not using a notebook)
├─ charts/
│   └─ age_histogram.png
│   └─ survival_by_class.png
│   └─ age_vs_fare_scatter.png
│   └─ correlation_heatmap.png
├─ REPORT.md               (findings + recommendation)
├─ SELF_EVALUATION.md      (completed rubric – see next section)
└─ README.md               (1-paragraph summary + time taken)
```

Step 3 — Commit properly

- Commit in stages that reflect your real progress (setup → NumPy → Pandas → Matplotlib → report). A single giant commit at the end is a red flag to reviewers, not a shortcut.
- Do not hardcode any local file paths — your notebook must run top-to-bottom on a fresh clone, using only the dataset URL provided.

Step 4 — Fill in your README

Your `README.md` should contain, at minimum:

- One paragraph summarizing what you did
- The **actual time you took**, stated honestly — this is used to calibrate the exercise, not to penalize you
- A link to your completed `REPORT.md` and `SELF_EVALUATION.md`

Step 5 — Share your link

Once your repository is public and complete, share it using this exact format:

Repo: [your GitHub link] — Total score: __/70 — Time taken: __ minutes — Weakest area: [section name]

That one line communicates more to a reviewer than a polished notebook with no self-awareness attached.

Self-Evaluation Rubric

Score yourself honestly on each row **before** sharing your repository. This is not graded by anyone else — it is a mirror. Copy this rubric into your `SELF_EVALUATION.md` with your scores filled in.

A. NumPy — out of 15

Criterion	0 — Missing	1 — Partial	2 — Solid	3 — Strong
Array conversion & basic stats	Not done	Used pandas instead of NumPy funcs	Correct NumPy stats, ignores NaN	Same, plus explains why NaN handling matters
Boolean masking	Not done	Mask created but not used correctly	Mask correctly filters data	Mask reused efficiently across tasks
<code>np.where</code> conditional labeling	Not done	Only two categories handled	All three categories correct	Clean, vectorized, no loops anywhere
NaN replacement via <code>np.where</code>	Not done	Used <code>.fillna()</code> instead	Correct <code>np.where</code> replacement	Verified with before/after NaN count
Code cleanliness (no unnecessary loops)	Loops everywhere	Some loops remain	Mostly vectorized	Fully vectorized, no <code>for</code> loops on data

Your score: ____ / 15

B. Pandas — out of 20

Criterion	0 — Missing	1 — Partial	2 — Solid	3-4 — Strong
Missing data strategy	Not addressed	Dropped everything blindly	Reasonable fill/drop choices	Choices justified in writing, per column
Feature engineering	Not done	One column created	Both columns correct	Both correct + sensible bucket boundaries
Business question 1 (class + gender)	Not answered	Wrong grouping	Correct result	Correct + one sentence interpretation
Business question 2 (family size)	Not answered	Wrong grouping	Correct result	Correct + one sentence interpretation
Business question 3 (port vs fare)	Not answered	Wrong grouping	Correct result	Correct + one sentence interpretation
Pivot table (class × age group)	Not done	Incomplete/wrong	Correct pivot table	Correct + a comment on the most notable cell

Your score: ____ / 20

C. Matplotlib / Visualization — out of 16

Criterion	0 — Missing	1 — Partial	2 — Solid	3-4 — Strong
Histogram (Age)	Not done	No labels/title	Labeled correctly	Labeled + bin choice justified
Bar chart (survival by class)	Not done	No labels/title	Labeled correctly	Labeled + clear takeaway stated
Scatter plot (Age vs Fare)	Not done	No color/marker distinction	Distinguishes survival status	Distinguishes + legend + clean layout
Heat map (correlation)	Not done	Missing annotations	Annotated, readable	Annotated + one sentence on strongest correlation

Your score: ____ / 16

D. Report Quality — out of 12

Criterion	0 — Missing	1 — Partial	2 — Solid	3-4 — Strong
Top 3 insights	Not written	Vague, no numbers	Specific, number-backed	Specific + tied directly to a chart/table
Recommendation	Not written	Generic	Actionable, data-tied	Actionable + acknowledges a limitation
Next steps	Not written	Vague	One clear next step	Two to three well-reasoned next steps

Your score: ____ / 12

E. GitHub Hygiene — out of 7

Criterion	Points
Repo is public and link works	1
Folder structure matches spec	1
Charts saved as actual <code>.png</code> files	1
<code>README.md</code> present with time-taken stated	1
Commit history shows real progression	1
No hardcoded local file paths	1
Notebook runs top-to-bottom without errors on a fresh clone	1

Your score: ____ / 7

Total Score & Self-Placement

Total: ____ / 70

Score range	Where you likely stand
60-70	Ready for junior data analyst / data science interviews — polish your storytelling next
45-59	Solid fundamentals; tighten up whichever section scored lowest before applying
30-44	Core syntax is there but workflow and speed need practice — redo this assignment in a week without looking at old code
Below 30	Revisit NumPy, Pandas, and Matplotlib fundamentals individually before attempting scenario-based work again

Good luck — remember, this activity measures readiness, not perfection. Submit honestly and you'll get far more value from it.